

Bracing the walls

>>Making your web applications more secure> by Jait Dixit

According to a recent report by Cenzic, an application security intelligence platform, 99 per cent of production application it tested had at least one security vulnerability with a median of 13 vulnerabilities per app. With increased adaptation of web-based technologies by businesses, along with increased utilization of cloud and mobile infrastructure, these figures may become more bleak in times to come. You may think that your application will not be as significant as high profile clients like The New York Times, Bank of America, and the US Federal Reserve, but Aesop's teaching of finding safety in insignificance doesn't hold much weight here. The hackers are in fact interested in your website's visitors. They primarily seek to compromise and infect all the end-users accessing your website by subjecting them to a variety of attacks, and in this case – every website counts.

In this article we'll discuss some of the most common security vulnerabilities in web applications and what you can do to protect yourself. One thing to point out, is that the method of applying these solutions will change based on your platform, programming language and framework used. With so many permutations and combinations, we'll not be able to explain every solutions in detail – but most of the language and framework wikis have dedicated pages on security along with excellent help and support available on forums, IRC and Stack Overflow.

SSH Authentication and SFTP

The very first advice here is to stop using passwords for both these services. Instead use public-private keypairs to authenticate a particular user. When you want to authorize yourself or any other user then you only have to put that public key in the `authorized_keys` file in an account on the server and the server allows anyone knowing the private key that's coupled to that public key to log into the server. In case you or other user loses his/her private key then you only need to remove the offending



Up your website security now



Safe websites lead to building user trust. So plug those holes.

public key from the server and generate a new one from scratch and authorize that key.

Another suggestion would be to stop using simple FTP and rather use SFTP and/or rsync to store and retrieve files from the server. SFTP essentially encrypts your datastream providing you with security at transport layer. OpenSSH and thus both SSH and SFTP support AES-256, 3DES and Blowfish so barring some overnight breakthrough in quantum computer, attackers will not be able to decrypt your data stream till the sun turns into a red giant.

Up-to-date production environment

The modus operandi of a typical hacker involves finding which version of operating system, language runtime and/or framework you're using and then execute a specific set of exploits which will allow them to compromise the system. The most effective way of protecting yourself from such attacks is to keep every piece of software up-to-date, this will give you a twofold advantage of better performance, new features and added security.

Obviously, this shouldn't be done at the cost of jeopardizing the applications functionality or feature. If your application doesn't have any esoteric dependency(s) which require you to use legacy or older version of underlying software, then it makes all the sense in the world to use the very latest versions of such software.

Use SSL/HTTPS

Although not always feasible, it is always better for security that you deploy your website using HTTPS instead of HTTP. This will make sure that user credentials and other data is not travelling down the line in plain text for everyone to see. Shifting to SSL/HTTPS sitewide means that you should serve everything over HTTPS – HTML, CSS, images, Javascript, everything. If you plan on doing then then we'd also suggest that you go ahead and